

Aletheia:
A Medical Question-Answering
Retrieval-Augmented Generation System

Fady A Eid

CSCI E-222: Foundations of Large Language Models

Harvard Extension School, Spring 2026

Final Project Report

Video presentation: [YouTube URL placeholder]

Abstract

General-purpose language models hallucinate on medical questions, producing plausible but ungrounded answers. Aletheia tests whether retrieval-augmented generation over a curated medical corpus reduces this failure mode. The system uses the MedCPT bi-encoder to retrieve the top three passages from MedQuAD (16,406 question-answer pairs from twelve NIH websites), reorders them using the lost-in-the-middle heuristic, and conditions BioMistral-7B-SLERP on the result under greedy decoding. Evaluation on a 1,641-question test split gives Top-3 Accuracy 0.4302, MRR@10 0.3567, ROUGE-L F1 0.2735, and BERTScore F1 0.2041. A 50-question ablation isolates retrieval’s contribution: adding retrieval improves ROUGE-L F1 by +0.0990 and BERTScore F1 by +0.1226. The per-category breakdown shows the gap is largest for frequency questions (short formulaic answers) and smallest or negative for symptoms, where the generator’s parametric knowledge competes with retrieved context. The system runs on a Dell Pro Max with GB10 (NVIDIA Grace Blackwell Superchip, 128 GB unified memory), with bit-exact reproducibility via fixed seeds and deterministic algorithms.

1. Introduction and Problem Statement

A patient looking up symptoms of type 2 diabetes today is likely to receive an answer from a large language model. The answer will sound confident. Whether it is correct is a separate question. The MedHallu benchmark confirms the failure mode is real: state-of-the-art models including GPT-4o and medically fine-tuned LLMs scored as low as 0.625 F1 on detecting hard-category medical hallucinations [1]. When the cost of a wrong answer is a misinformed health decision, “confidently plausible” is not good enough.

Retrieval-augmented generation, introduced by Lewis et al. [2], addresses this failure mode by grounding generation in external documents. Instead of relying on what the model memorized during pretraining, the system retrieves relevant passages from a curated corpus at inference time and conditions the answer on them. For medical question answering, the benefit is twofold: the generator stops fabricating facts it never saw, and every answer becomes traceable back to a source document a clinician or user can verify.

Aletheia implements this idea over MedQuAD, a collection of 16,406 medical question-answer pairs sourced from twelve National Institutes of Health websites [3]. The system uses the MedCPT bi-encoder to retrieve the top three passages per question, applies lost-in-the-middle reordering, and uses BioMistral-7B-SLERP to generate the final answer under greedy decoding. The central

experimental question is whether adding retrieval to a generator trained on medical text measurably improves answer quality, and where that improvement comes from.

The remainder of this report follows the project requirements. Section 2 describes the data and preprocessing. Section 3 covers the methods. Section 4 documents the implementation environment. Section 5 reports results on five metrics over a test split of 1,641 questions, plus a controlled ablation on 50 questions. Section 6 discusses what worked and what did not. Section 7 closes with limitations, including disclosure of the evaluation’s design choice around gold leakage.

2. Data Description and Preprocessing

The dataset is MedQuAD, released by Ben Abacha and Demner-Fushman in 2019 [3], containing question-answer pairs scraped from twelve consumer-health websites operated by the National Institutes of Health. The full release has 47,441 rows, but 31,034 had their answer fields stripped from the public mirror for copyright reasons (the GARD, MPlusHerbsSupplements, ADAM, and MPlusDrugs subsets are sourced from MedlinePlus, which does not permit redistribution).

Two preprocessing filters apply. First, rows with empty or whitespace-only answers are dropped, leaving 16,407 rows; embedding an empty string produces a vector that means almost nothing and would silently corrupt retrieval. Second, the `question_type` column has 16 values, but support groups has exactly one row. Hugging Face’s stratified splitter refuses to run when a class has fewer than two members, so this singleton is dropped, leaving 16,406 rows across 15 classes.

After encoding `question_type` from strings to integers (required by the stratified splitter), the data is partitioned into 13,124 rows in `train_pool`, 1,641 in `val`, and 1,641 in `test`, using seed 42 so reruns produce identical splits. Aletheia does not fine-tune any model, so `train_pool` is saved but never read again. The full 16,406 rows are also saved under the name `corpus` because they serve as the retrieval index in Section 3, a design choice discussed in Section 7.

Two checks confirm the data is ready. Tokenizing every answer with the `MedCPT-Article` tokenizer gives a median of 173 tokens, mean of 249.5, and maximum of 5,033, with a long right tail. 85.3% of answers fit under the 400-token chunking cap chosen for Section 3, 5.9% fall between 400 and 512 (`MedCPT-Article`’s input ceiling), and 8.8% exceed 512. That last group would be silently truncated by the embedder without chunking. Stratification held: the worst-case drift across any class is 0.06 percentage points, well under one. Class imbalance, however, persists: information is 27.64% of the corpus, symptoms is 16.75%, while complications and stages together account for

less than 1%. The rarest classes end up with single-digit test samples, so their per-class metrics are unstable (revisited in Section 6).

3. Models and Methods

The pipeline runs in seven stages: chunking, embedding, indexing, retrieval, reordering, prompt construction, and generation.

3.1 Chunking

Long answers exceed the encoder’s input ceiling. The chosen splitter is LangChain’s `RecursiveCharacterTextSplitter` with a 400-token cap and 50-token overlap, measured against the `MedCPT-Article` tokenizer. The recursive splitter tries natural boundaries first (paragraphs, then line breaks, then spaces) and only falls back to cuts at the character level when the cap forces it. The alternative considered was Hugging Face’s tokenizer-based chunking with `return_overflowing_tokens=True` and `stride`, which chops at fixed positions regardless of word or sentence boundaries: the canonical Hugging Face example with `max_length=12` and `stride=7` cuts xylophones into xylo and phones [4]. For medical text where terms like hyperthyroidism matter, that behavior produces subtokens no encoder was trained to embed coherently. Applied to the 16,406 filtered answers, the recursive splitter produces 20,604 chunks; the longest answer splits into 16.

3.2 Embedding

The embedder is `MedCPT-Article-Encoder`, a BERT-base model with 109.5 million parameters from the National Center for Biotechnology Information [5]. MedCPT is a bi-encoder: `MedCPT-Article` for documents and `MedCPT-Query` for queries, both initialized from PubMedBERT and trained together on 255 million query-article pairs from PubMed search logs between 2020 and 2022 [5]. Because the encoders are trained jointly, their outputs share a 768-dimensional space, so query-to-article inner product is a meaningful similarity score. For each chunk, the encoder takes the [CLS] hidden state from the final layer and applies L2 normalization. The choice of [CLS] over mean pooling matters: mean pooling does not match what `MedCPT-Query` was trained to produce and degrades retrieval silently. MedCPT was chosen over generic alternatives (sentence-transformers, E5, BGE-M3) because its training distribution (real PubMed search queries) matches MedQuAD’s question style; the tradeoff is that MedCPT is limited to English and biomedical text, both of which fit Aletheia.

3.3 Indexing

The 20,604 vectors load into a FAISS `IndexFlatIP` index from `faiss-cpu` [6]. Flat means exact brute-force search; IP means inner product, which on L2-normalized vectors is cosine similarity. At 20,604 vectors, brute-force scan completes in milliseconds and avoids approximate-index recall loss. `faiss-cpu` is forced by hardware: the GB10’s GPU is aarch64, and no `faiss-gpu` wheel exists for that architecture.

3.4 Retrieval and reordering

At inference, the question is tokenized at `max_length=64` (matching MedCPT’s model card example), passed through `MedCPT-Query`, and L2-normalized into a 768-dimensional vector. FAISS returns the top three chunks. Top-3 was chosen because more chunks compound the lost-in-the-middle effect, where decoder-only models attend more strongly to the start and end of context than the middle [7], and because three 400-token chunks plus the prompt template plus a 256-token generation budget fit comfortably within BioMistral’s 32,768-token context window (`max_position_embeddings`), which the notebook enforces at runtime via the `CTX_LIMIT` check in cell 67. BioMistral inherits Mistral-7B-v0.1’s 4,096-token sliding-window attention, so although the model accepts longer inputs, each attention layer only reaches back 4,096 tokens. At the prompt sizes used here this distinction does not matter, but it would for tasks placing key information thousands of tokens apart. Even at `k=3`, retrieved chunks are reordered following Liu et al. [7]: the most similar chunk goes to the position closest to the question (the end), the next most similar goes at the start, and the third fills the middle.

3.5 Prompt and generation

The prompt has a fixed structure: a system instruction telling the model to answer only from the provided context, the three reordered chunks under a Context: header, the question, and an Answer: cue. The system instruction also permits the model to respond “I do not know” if the context lacks the answer. The generator is **BioMistral-7B-SLERP**, a 7.24 billion parameter merged model from Labrak et al. [8]. The SLERP recipe interpolates two source models (BioMistral-7B, which is Mistral-7B-Instruct-v0.1 further pretrained on PubMed Central, and Mistral-7B-Instruct-v0.1 itself) across 32 transformer layers, weighting attention layers toward BioMistral and MLP layers toward Mistral-Instruct. Generation uses greedy decoding (`do_sample=False`, `num_beams=1`, `max_new_tokens=256`) for full determinism. Attention uses PyTorch’s scaled dot-product (`sdpa`) because flash-attention is unavailable on aarch64. Weights load at BF16, fitting the 14.48 GB of merged weights comfortably in the GB10’s 128 GB unified memory.

4. Implementation Details

4.1 Environment

The system runs on a Dell Pro Max workstation with the NVIDIA GB10 Grace Blackwell Superchip (128 GB unified memory, aarch64, DGX OS), CUDA 13, and Python 3.12. aarch64 forces specific dependency choices: PyTorch from the cu130 wheel index, **faiss-cpu** rather than GPU FAISS, and **sdpa** attention rather than flash-attention. The library stack is transformers 4.46+, datasets 3.0+, **faiss-cpu** 1.13.2, langchain-text-splitters, rouge-score, and bert-score. Exact versions are pinned in `requirements.txt`. The three models (MedCPT encoders, BioMistral-7B-SLERP, RoBERTa-large for BERTScore) total roughly 16 GB and download on first run.

4.2 Reproducibility

Every random source is seeded with 42: Python random, NumPy, PyTorch CPU and CUDA generators, and Transformers via `set_seed`. PyTorch's `torch.use_deterministic_algorithms(True, warn_only=True)` forces deterministic CUDA ops where available. `CUBLAS_WORKSPACE_CONFIG=:16:8` is set before any CUDA call (setting it after has no effect). cuDNN's deterministic mode is enabled and benchmark mode disabled. Together, these settings make embedding, retrieval, and greedy decoding bit-exact reproducible across runs on the same hardware.

4.3 Run instructions

The full pipeline lives in `aletheia.ipynb`. To reproduce:

1. Install:

```
pip install -r requirements.txt --extra-index-url https://download.pytorch.org/whl/cu130
```

2. Run cells top to bottom. The notebook handles dataset download, preprocessing, embedding, indexing, retrieval, generation, and evaluation.
3. Pre-computed embeddings (`embeddings.npy`, 63 MB) are available at the GitHub repo: <https://github.com/fxeid/aletheia/raw/main/data/embeddings.npy>. Place in `data/` to skip the embedding step.

End-to-end runtime on the GB10 is roughly 90 to 140 minutes, dominated by RAG generation (1,641 questions, roughly 3 seconds each, about 80 minutes).

4.4 Submission package

The ZIP contains the notebook, `requirements.txt`, `README.md`, and a 200-row MedQuAD sample. The full dataset is handled by `load_dataset`. The embeddings file is hosted externally because it exceeds the 10 MB ZIP limit.

4.5 Reproducing on Google Colab

For readers without access to a Dell Pro Max with GB10, the experiments reproduce on Google Colab with one architectural caveat. The free-tier T4 GPU does not support BF16 (T4 is Turing architecture, predating BF16 hardware support), so the free tier requires 4-bit quantization, which will not exactly match the reported BF16 numbers. Colab Pro provides A100 or L4 GPUs, both BF16-capable, which reproduce the numbers faithfully.

Verified package compatibility. The following package versions are known to work on Colab x86_64 Linux: `transformers>=4.46,<5.0`, `datasets>=3.0,<5.0`, `faiss-cpu==1.13.2`, `langchain-text-splitters`, `rouge-score`, `bert-score`, `accelerate`, and for the free tier, `bitsandbytes`. Pinning `transformers<5.0` is important because version 5.0 (released January 2026) changed the dtype default and removed the `load_in_4bit=True` shortcut; the notebook code targets the 4.x API. The `faiss-cpu 1.13.2` wheel for `manylinux_x86_64` is published on PyPI and installs cleanly on Colab.

Path A: Colab Pro (A100 or L4, BF16-native). Select Runtime, then Change runtime type, then A100 GPU. The setup cell at the start of the notebook becomes:

```
!pip install -q "transformers>=4.46,<5.0" "datasets>=3.0,<5.0" \
    faiss-cpu==1.13.2 langchain-text-splitters rouge-score bert-score accelerate
```

All other notebook cells run unchanged. The existing seed and determinism block handles reproducibility. End-to-end runtime on A100 is approximately 60 to 90 minutes, slightly faster than the GB10.

Path B: Colab free tier (T4, 4-bit quantization). T4 lacks BF16 support and has only 15 GB usable VRAM, so BioMistral-7B at full BF16 precision will not fit. Two adaptations are required. First, install `bitsandbytes` and replace the generator-loading cell with:

```
from transformers import BitsAndBytesConfig, AutoModelForCausalLM
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
)

generator = AutoModelForCausalLM.from_pretrained(
    "BioMistral/BioMistral-7B-SLERP",
```

```

    quantization_config=bnb_config,
    device_map="auto",
)

```

Second, change the MedCPT embedding dtype from `torch.bfloat16` to `torch.float16`. Generation metrics under 4-bit quantization will not match the reported BF16 numbers exactly; quantization typically degrades ROUGE-L F1 and BERTScore F1 by a few percentage points, but the qualitative findings (RAG above no-RAG, frequency easiest, symptoms hardest) should hold.

Pre-computed embeddings. To skip the embedding step on first run, download `embeddings.npy` (63 MB) from the link in Section 4.3 and place it in `data/`. This is more useful on Colab than on the GB10 because re-embedding on a T4 takes roughly 15 to 20 minutes versus 8 minutes on the GB10’s Blackwell GPU. Mounting Google Drive (from `google.colab import drive`; `drive.mount('/content/drive')`) is recommended for storing this file across sessions, since Colab’s runtime storage is wiped when the session ends.

5. Experiments and Results

5.1 Retrieval metrics on n=1,641

For each test question, the retriever returns the top 10 chunks.

Metric	Value
Top-1 Accuracy	0.2261
Top-3 Accuracy	0.4302
Top-5 Accuracy	0.5436
MRR@5	0.3389
MRR@10	0.3567

The gap from Top-3 (0.4302) to Top-5 (0.5436) suggests increasing depth could improve coverage, at the cost of more context and stronger exposure to lost-in-the-middle effects. Top-3 was kept fixed for generation. “Top-k Accuracy” is used here rather than “Recall@k” because with a single gold document per query, the two metrics are mathematically equivalent.

5.2 Generation metrics on n=1,641

Each generated answer is compared to the MedQuAD reference.

Metric	Value
ROUGE-L F1	0.2735
BERTScore F1	0.2041

Both metrics are far from 1.0, which is expected. MedQuAD references are long paragraphs scraped from NIH webpages; BioMistral’s outputs are shorter and phrased differently even when factually equivalent. ROUGE-L penalizes any rewording. Rescaled BERTScore at 0.2041 indicates moderate semantic alignment above random pairs but below paraphrase quality.

5.3 Ablation: RAG versus no-RAG on n=50

The same generator runs against the same 50 questions with and without retrieved context.

Metric	RAG	no-RAG	Delta
ROUGE-L F1	0.2832	0.1842	+0.0990
BERTScore F1	0.2312	0.1086	+0.1226

Retrieval improves both metrics by a clear margin. The direction is unambiguous; the magnitude at n=50 is noisy. The +0.1226 BERTScore improvement is notable because rescaled BERTScore is centered on a random-pair baseline, so the delta represents absolute movement in semantic alignment.

5.4 Per-question-type breakdown

Selected rows from the 15-class breakdown (full table in notebook Section 8.5):

Type	n	ROUGE-L F1	BERTScore F1
frequency	112	0.4957	0.5058
information	453	0.3199	0.2869
inheritance	144	0.2368	0.2743
treatment	245	0.2117	0.0877
symptoms	275	0.1532	-0.0111
stages	7	0.0651	-0.0901

Three observations. Frequency questions are easiest: short formulaic answers (“Approximately X people in Y”) match retrieved chunks nearly verbatim. Symptoms produces negative rescaled BERTScore at large sample size (n=275), a real finding discussed in Section 6. Rare categories (n less than 30: considerations, prevention, stages, complications) have estimates dominated by sampling noise and should not be over-interpreted.

6. Discussion

6.1 What worked

The ablation gap is the clearest result. Retrieval moves BERTScore F1 from 0.1086 (no-RAG, near baseline) to 0.2312 (RAG), a delta of +0.1226 that holds across both lexical and semantic measures. This is the difference between essentially-random medical-sounding text and content recognizably aligned with NIH source material. The per-class breakdown confirms the mechanism: categories with short formulaic answers (frequency, outlook, research, exams and tests) all score above the overall mean. These are cases where retrieval finds the exact answer and the generator copies it nearly verbatim.

6.2 What did not work

Symptoms questions produce negative rescaled BERTScore F1 (-0.0111) at $n=275$. Two hypotheses. First, BioMistral’s parametric knowledge of common symptoms may override retrieved context: the model “knows” the symptoms of diabetes from pretraining and produces them in its own phrasing rather than copying NIH’s phrasing. This would explain negative BERTScore: the model produces different text from gold even when both are factually correct. Second, MedQuAD’s symptoms answers vary widely in length and structure across conditions, so any single phrasing the model converges to will mismatch the average gold. Neither hypothesis is testable from the current evaluation; both would benefit from manual qualitative review of a symptoms sample (future work).

6.3 Lessons learned

Mean pooling versus [CLS] pooling is a silent bug: both produce plausible-looking vectors, but mean pooling does not match what MedCPT-Query was trained to produce. The model card’s example uses [CLS]; deviating is a trap. Deterministic CUDA requires CUBLAS_WORKSPACE_CONFIG set before any CUDA call, not after; setting it in a later cell has no effect on prior operations. ROUGE-L and BERTScore correlate weakly with expert factual judgments on medical text; MORQA [9] demonstrates that LLM-based evaluators outperform traditional metrics on medical open-ended QA. The metrics here are the offline-reproducible compromise the rubric rewards, not estimates of clinical correctness.

6.4 Possible extensions

Three concrete next steps: add MedCPT’s cross-encoder reranker [5] on top of the bi-encoder to improve Top-1 Accuracy; evaluate on a temporal split where test queries reference documents added after a corpus freeze date, directly measuring generalization to unseen documents; run an LLM-as-judge evaluation following MORQA’s protocol, measuring factual correctness rather than text similarity.

7. Limitations and Responsible-Use Considerations

7.1 Gold leakage by design

The FAISS index contains chunks from all 16,406 corpus documents. The 80/10/10 split partitions queries, not documents, so test questions’ gold answers exist in the index. This is gold leakage by design, and it must be disclosed because it affects how the numbers should be read. The setup mirrors how production RAG systems are evaluated: a deployment serves the same knowledge base to every user, so the relevant question is “given a static corpus, can retrieval find the right document?” A genuine train-pool-only index would not produce a more useful number; test gold would not be in the index by construction, so Top-k Accuracy would collapse toward zero. To make a train-pool-only setup meaningful, the evaluation would need to redefine “correct retrieval” as finding semantically-related documents, requiring a new gold standard. The project deliberately uses the standard metric (gold-document recall). The strongest reading of the reported numbers is comparative: the RAG-versus-no-RAG ablation, per-category breakdown, and MRR distribution describe relative performance under different conditions, and these comparisons hold under any indexing choice. Absolute Top-k values are calibrated to the closed-corpus setting.

7.2 Metric limitations

ROUGE-L and BERTScore measure surface lexical and contextual semantic similarity. Neither measures clinical correctness. MORQA [9] demonstrates traditional automatic metrics correlate weakly with expert judgments on medical QA; LLM-based evaluators show substantially stronger correlation. The metrics here are appropriate for offline reproducible evaluation, not estimates of clinical reliability.

7.3 Sample size limits

The test split has 1,641 questions across 15 classes. The four rarest types (complications, stages, prevention, considerations) together account for 57 questions. Per-class metrics for these are dominated by sampling noise.

7.4 Responsible use

Aletheia is a research prototype on a static benchmark, not a clinical tool. It has no access to up-to-date literature beyond MedQuAD’s snapshot, does not personalize answers, and does not flag cases where retrieved context disagrees with parametric knowledge. The “I do not know” refusal is permitted but the model is not trained on uncertainty calibration; a user cannot distinguish a grounded answer from a hallucinated one without consulting the source. For any medical decision, the appropriate use is verification: every answer should be cross-checked against the `document_url` field preserved in MedQuAD’s metadata and against a clinician’s judgment. The system is suitable for educational or research contexts and for ablation studies, not for direct patient-facing use.

References

- [1] S. Pandit, J. Xu, J. Hong, Z. Wang, T. Chen, K. Xu, and Y. Ding, “MedHallu: A Comprehensive Benchmark for Detecting Medical Hallucinations in Large Language Models,” in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Suzhou, China, 2025, pp. 2858-2873.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020.
- [3] A. Ben Abacha and D. Demner-Fushman, “A Question-Entailment Approach to Question Answering,” BMC Bioinformatics, vol. 20, no. 1, p. 511, Oct. 2019.
- [4] Hugging Face documentation, “Tokenizer: returning overflowing tokens with stride,” available at https://huggingface.co/docs/transformers/main_classes/tokenizer, accessed May 2026.
- [5] Q. Jin, W. Kim, Q. Chen, D. C. Comeau, L. Yeganova, W. J. Wilbur, and Z. Lu, “MedCPT: Contrastive Pre-trained Transformers with Large-scale PubMed Search Logs for Zero-shot Biomedical Information Retrieval,” Bioinformatics, vol. 39, no. 11, btad651, Nov. 2023.

- [6] J. Johnson, M. Douze, and H. Jegou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
- [7] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157-173, 2024.
- [8] Y. Labrak, A. Bazoge, E. Morin, P.-A. Gourraud, M. Rouvier, and R. Dufour, “BioMistral: A Collection of Open-Source Pretrained Large Language Models for Medical Domains,” in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, 2024.
- [9] W. Yim, A. Ben Abacha, Z. Yu, R. Doerning, F. Xia, and M. Yetisgen, “MORQA: Benchmarking Evaluation Metrics for Medical Open-Ended Question Answering,” *arXiv:2509.12405*, Sep. 2025.